

```
In [62]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import sqlite3
```

```
In [63]: conn=sqlite3.connect('C:/Users/OWNER/Downloads/customer_churn.db')

sql_query="""
        SELECT name
        FROM sqlite_master
        WHERE type='table';
    """

tables=pd.read_sql(sql_query, conn)
#create table from each table
for table_name in tables['name']:
    df=pd.read_sql(f"SELECT * FROM {table_name}",conn)
    globals()[f"df_{table_name}"]=df
    print(f"created dataframe:df_{table_name}")

conn.close()
```

created dataframe:df_db_customer
created dataframe:df_db_subscription
created dataframe:df_db_support

```
In [64]: df_db_customer.head()
```

```
Out[64]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead>
<tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody>
<tbody></tbody>
```

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None

```
In [65]: df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   customerid  21 non-null    str
 1   name        21 non-null    str
 2   country     18 non-null    str
 3   state       21 non-null    str
 4   gender      21 non-null    str
 5   dob         21 non-null    str
 6   interests   4 non-null     str
 7   pincode     0 non-null     object
dtypes: object(1), str(7)
memory usage: 1.4+ KB
```

```
In [5]: #a.rename name to customername
        #b.drop columns=customer and pincode
        #c.change datatype to DOB
        #d.fix missing values of country
```

```
In [66]: df_db_customer.rename(columns={'name': 'customer_name'}, inplace=True)
         print(df_db_customer)
```

	customerid	customer_name	country	state	gender	\
0	0002-ORFBO	keshav	India	Maharashtra	Male	
1	0003-MKNFE	raghav	India	Karnataka	Male	
2	0004-TLHLJ	lalita	India	Delhi	Female	
3	0011-IGKFF	mohan	India	Nagaland	Male	
4	0013-EXCHZ	mira	India	Delhi	Female	
5	0013-MHZWF	durga	NaN	Delhi	Women	
6	0013-SMEOE	mina	India	Meghalaya	Female	
7	0014-BMAQU	madan	India	Rajasthan	Male	
8	0015-UOCOJ	maya	NaN	Kathmandu	Women	
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	
10	0017-DINOC	shiva	India	Maharashtra	Men	
11	0017-IUDMW	rangadevi	India	Karnataka	Female	
12	0018-NYROU	chitra	NaN	Telangana	Female	
13	0019-EFAEP	raju	India	Meghalaya	Female	
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	
15	0020-INWCK	parvati	India	Delhi	Female	
16	0020-JDNXP	rikim	India	Meghalaya	Female	
17	0021-IXXGC	vishakha	India	Rajasthan	Female	
18	0022-TCJCI	raghvendra	India	Telangana	Male	
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	
20	0023-UYUPN	sudevi	India	Maharashtra	Women	

	dob	interests	pincode
0	1982-04-12 00:00:00	travel	None
1	1995-11-23 00:00:00	NaN	None
2	1978-02-15 00:00:00	movie	None
3	2001-08-30 00:00:00	NaN	None
4	1990-05-05 00:00:00	drama	None
5	1988-12-10 00:00:00	NaN	None
6	1976-09-21 00:00:00	NaN	None
7	1999-03-14 00:00:00	NaN	None
8	1985-07-07 00:00:00	NaN	None
9	1993-10-29 00:00:00	NaN	None
10	1997-01-22 00:00:00	NaN	None
11	1981-06-18 00:00:00	NaN	None
12	2004-12-01 00:00:00	NaN	None
13	1992-04-25 00:00:00	NaN	None
14	1979-11-11 00:00:00	NaN	None
15	1986-02-28 00:00:00	job	None
16	1994-08-19 00:00:00	NaN	None
17	2000-09-02 00:00:00	NaN	None
18	1983-12-30 00:00:00	NaN	None
19	1991-05-14 00:00:00	NaN	None
20	1977-10-06 00:00:00	NaN	None

```
In [67]: df_db_customer.drop(df_db_customer.columns[-2:],axis=1)
#df_db_customer.drop(columns=['interests', 'pincode'], inplace=True)
```

```
Out[67]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th>
<th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th>
<th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody>
</tbody>
```

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00
0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00
0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00
0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00
0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00
0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00
0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00
0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00
0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00
0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00
0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00
0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00

```
In [68]: df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customerid      21 non-null    str
1   customer_name   21 non-null    str
2   country         18 non-null    str
3   state           21 non-null    str
4   gender          21 non-null    str
5   dob             21 non-null    str
6   interests       4 non-null     str
7   pincode         0 non-null     object
dtypes: object(1), str(7)
memory usage: 1.4+ KB
```

```
In [69]: df_db_customer['dob']=pd.to_datetime(df_db_customer['dob'])
```

```
In [70]: #df_db_customer['gender'].unique()
df_db_customer['gender']=df_db_customer['gender'].replace({'Men':'Male','Women':'Female'})
df_db_customer['gender'].unique()
```

```
Out[70]: <StringArray>
['Male', 'Female']
Length: 2, dtype: str
```

```
In [71]: df_db_customer['country'].fillna("not fill")
```

```
Out[71]: 0      India
1      India
2      India
3      India
4      India
5  not fill
6      India
7      India
8  not fill
9      Nepal
10     India
11     India
12  not fill
13     India
14     India
15     India
16     India
17     India
18     India
19     India
20     India
Name: country, dtype: str
```

```
In [72]: state_country_mapping=df_db_customer.dropna(subset=['country']).set_index('state')[
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['state'])
```

```
In [73]: df_db_customer[['country','state']]
```

```
Out[73]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>country</th> <th>state</th> </thead> <tbody> <th>0</th> <th>1</th>
<th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th>
<th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th>
<th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th>
</tbody> <tbody> </tbody>
```

India	Maharashtra
India	Karnataka
India	Delhi
India	Nagaland
India	Delhi
India	Delhi
India	Meghalaya
India	Rajasthan
Nepal	Kathmandu
Nepal	Kathmandu
India	Maharashtra
India	Karnataka
India	Telangana
India	Meghalaya
India	Uttar Pradesh
India	Delhi
India	Meghalaya
India	Rajasthan
India	Telangana
India	Uttar Pradesh
India	Maharashtra

```
In [74]: df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    str
1   subscription_start_date 21 non-null    str
2   subscription_type       21 non-null    str
3   renewal_date           21 non-null    str
4   plan_type              21 non-null    str
5   contract_type          21 non-null    str
6   cancellation_date       6 non-null     str
7   cancellation_reason     6 non-null     str
8   monthly_charges        21 non-null    float64
9   cltv                   21 non-null    int64
10  churn_score            21 non-null    int64
dtypes: float64(1), int64(2), str(8)
memory usage: 1.9 KB
```

```
In [75]: print(tables['name'].tolist())
```

```
['db_customer', 'db_subscription', 'db_support']
```

```
In [76]: date_col=['subscription_start_date','renewal_date','cancellation_date']
df_db_subscription[date_col]=df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    str
1   subscription_start_date 21 non-null    datetime64[us]
2   subscription_type       21 non-null    str
3   renewal_date           21 non-null    datetime64[us]
4   plan_type              21 non-null    str
5   contract_type          21 non-null    str
6   cancellation_date       6 non-null     datetime64[us]
7   cancellation_reason     6 non-null     str
8   monthly_charges        21 non-null    float64
9   cltv                   21 non-null    int64
10  churn_score            21 non-null    int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB
```

```
In [77]: df_db_support.head()
```

```
Out[77]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th>
<th>csat_score</th> <th>col_1</th> <th>comment</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>
```

0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN

```
In [78]: df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customerid      9 non-null     str
1   complaint_date  9 non-null     str
2   escalations     9 non-null     str
3   csat_score      9 non-null     int64
4   col_1           0 non-null     object
5   comment         4 non-null     str
dtypes: int64(1), object(1), str(4)
memory usage: 564.0+ bytes
```

```
In [79]: print(df_db_support.columns.tolist())
df_db_support.drop(columns=['col_1', 'comment'], inplace=True)
```

```
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

```
In [80]: df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customerid      9 non-null     str
1   complaint_date  9 non-null     str
2   escalations     9 non-null     str
3   csat_score      9 non-null     int64
dtypes: int64(1), str(3)
memory usage: 420.0 bytes
```

```
In [81]: df_db_support['complaint_date']=pd.to_datetime(df_db_support['complaint_date'])
df_db_support.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customerid      9 non-null     str
1   complaint_date  9 non-null     datetime64[us]
2   escalations     9 non-null     str
3   csat_score      9 non-null     int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 420.0 bytes

```

In [22]: *#feature engineering and data analysis*

In [82]: `df_db_subscription.info()`

```

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                      Non-Null Count  Dtype
---  ---
0   customerid                  21 non-null     str
1   subscription_start_date     21 non-null     datetime64[us]
2   subscription_type           21 non-null     str
3   renewal_date                21 non-null     datetime64[us]
4   plan_type                   21 non-null     str
5   contract_type               21 non-null     str
6   cancellation_date           6 non-null      datetime64[us]
7   cancellation_reason         6 non-null      str
8   monthly_charges             21 non-null     float64
9   cltv                        21 non-null     int64
10  churn_score                  21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB

```

In [83]: *#create a new column*

```

df_db_subscription['churn flag'] = np.where(
    df_db_subscription['cancellation_date'].notna(), 1, 0
)

df_db_subscription.head()

```

```
Out[83]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th>
<th>renewal_date</th> <th>plan_type</th> <th>contract_type</th>
<th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th>
<th>cltv</th> <th>churn_score</th> <th>churn flag</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>
```

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	12	0
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	34	0
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	8	0
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88	1

```
In [85]: df=(df_db_subscription
            .merge(df_db_subscription,on='customerid',how='left')
            .merge(df_db_support,on='customerid',how='left'))
```

```
In [86]: df.shape
```

```
Out[86]: (23, 26)
```

```
In [87]: df_db_subscription.shape
```

```
Out[87]: (21, 12)
```

```
In [28]: df_db_subscription['customerid'].nunique()
```

```
Out[28]: 21
```

```
In [29]: df_db_customer['customerid'].nunique()
```

```
Out[29]: 21
```

```
In [30]: df_db_support['customerid'].nunique()
```

```
Out[30]: 7
```

```
In [31]: df_db_support['customerid'].size
```

```
Out[31]: 9
```

```
In [88]: df_db_support['complaint_count']=df_db_support.groupby('customerid')['customerid'].size
```

```
In [89]: df_db_support
```

```
Out[89]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> <th>complaint_count</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> </tbody> <tbody></tbody>
```

```
0003-MKNFE 2024-08-28 N 60 2
```

```
0003-MKNFE 2024-08-28 Y 10 2
```

```
0013-EXCHZ 2024-01-20 Y 20 1
```

```
0013-MHZWF 2025-03-18 N 90 1
```

```
0013-SMEOE 2024-11-01 N 30 1
```

```
0017-IUDMW 2024-04-10 Y 25 1
```

```
0019-EFAEP 2024-09-27 Y 30 1
```

```
0022-TCJCI 2024-09-13 Y 10 2
```

```
0022-TCJCI 2024-09-14 N 90 2
```

```
In [90]: df_db_support=df_db_support.sort_values('complaint_date').drop_duplicates('customerid')
```

```
In [91]: df_db_support['customerid'].size
```

```
Out[91]: 7
```

```
In [92]: df=(df_db_subscription
            .merge(df_db_customer,on='customerid',how='left')
            .merge(df_db_support,on='customerid',how='left'))
```

```
In [93]: df.shape
```

```
Out[93]: (21, 23)
```

```
In [94]: df.to_csv('exported_churn_data.csv',index=False)
```

```
In [39]: #data analysis
        #a.churn rate
```

```
In [95]: import os

        print(os.getcwd())
```

C:\Users\OWNER\Desktop\python data jupyter

In [97]: `df.columns`

Out[97]: Index(['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'churn flag', 'customer_name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode', 'complaint_date', 'escalations', 'csat_score', 'complaint_count'], dtype='str')

In [98]: `churn_rate=df['churn flag'].mean()*10`
`print("churn Rate =",round(churn_rate,2), "%")`

churn Rate = 2.86 %

In [99]: `retation_rate=100-churn_rate`
`print("retation Rate =",round(retation_rate,2), "%")`

retation Rate = 97.14 %

In [100... `df.head()`

Out[100... `<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>`
`</th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th>`
`<th>renewal_date</th> <th>plan_type</th> <th>contract_type</th>`
`<th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th>`
`<th>cltv</th> <th>...</th> <th>country</th> <th>state</th> <th>gender</th>`
`<th>dob</th> <th>interests</th> <th>pincode</th> <th>complaint_date</th>`
`<th>escalations</th> <th>csat_score</th> <th>complaint_count</th> </thead> <tbody>`
`<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>`
`</tbody>`

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	...	India
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	India
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	...	India
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	...	India
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	...	India

`<p>5 rows × 23 columns</p>`



In [101... `churn_by_plan=(df.groupby('plan_type')['churn flag'].mean().mul(100).round(2))`

```
print(churn_by_plan)
```

```
plan_type
Basic      60.00
Premium    14.29
Standard   22.22
Name: churn flag, dtype: float64
```

```
In [102... df.columns
```

```
Out[102... Index(['customerid', 'subscription_start_date', 'subscription_type',
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
      'churn flag', 'customer_name', 'country', 'state', 'gender', 'dob',
      'interests', 'pincode', 'complaint_date', 'escalations', 'csat_score',
      'complaint_count'],
      dtype='str')
```

```
In [103... df.groupby('state').agg(
    revenue=('monthly_charges', 'sum'),
    users=('customerid', 'count'),
    churn=('churn flag', 'sum')
)
```

```
Out[103... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>revenue</th> <th>users</th> <th>churn</th> <th>state</th> <th></th>
<th></th> <th></th> </thead> <tbody> <th>Delhi</th> <th>Karnataka</th>
<th>Kathmandu</th> <th>Maharashtra</th> <th>Meghalaya</th> <th>Nagaland</th>
<th>Rajasthan</th> <th>Telangana</th> <th>Uttar Pradesh</th> </tbody> <tbody>
</tbody>

52.96  4  1
20.98  2  2
20.98  2  0
50.97  3  0
42.97  3  2
22.99  1  0
36.98  2  0
30.98  2  1
115.98  2  0
```

```
In [104... df.groupby('subscription_type').agg(
    revenue=('monthly_charges', 'sum'),
    users=('customerid', 'count')
)
```

```
Out[104... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>revenue</th> <th>users</th> <th>subscription_type</th> <th></th> <th>
</th> </thead> <tbody> <th>Organic</th> <th>Paid</th> <th>Refferal</th> </tbody>
<tbody></tbody>

145.91  9
174.94  6
74.94  6
```

```
In [105... #average revenue pr user
df['monthly_charges'].mean()
```

```
Out[105... np.float64(18.847142857142856)
```

```
In [106... #count of days user has used in service :cancelation date else current date
pd.Timestamp.today()
```

```
Out[106... Timestamp('2026-09-17 11:34:17.801384')
```

```
In [107... df.columns
```

```
Out[107... Index(['customerid', 'subscription_start_date', 'subscription_type',
'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
'churn flag', 'customer_name', 'country', 'state', 'gender', 'dob',
'interests', 'pincode', 'complaint_date', 'escalations', 'csat_score',
'complaint_count'],
dtype='str')
```

```
In [108... df.head(2)
```

```
Out[108... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th>
<th>renewal_date</th> <th>plan_type</th> <th>contract_type</th>
<th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th>
<th>cltv</th> <th>...</th> <th>country</th> <th>state</th> <th>gender</th>
<th>dob</th> <th>interests</th> <th>pincode</th> <th>complaint_date</th>
<th>escalations</th> <th>csat_score</th> <th>complaint_count</th> </thead> <tbody>
<th>0</th> <th>1</th> </tbody> <tbody></tbody>
```

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	...	India
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	India

<p>2 rows × 23 columns</p>



```
In [109... #calculate customer age
df['dob']=pd.to_datetime(df['dob'])
df['customer_age']=(pd.Timestamp.today().year-df['dob'].dt.year)
df[['dob','customer_age']].head()
```

```
Out[109... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>dob</th> <th>customer_age</th> </thead> <tbody> <th>0</th> <th>1</th>
<th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>
```

1982-04-12 44

1995-11-23 31

1978-02-15 48

2001-08-30 25

1990-05-05 36

```
In [110... df.columns
```

```
Out[110... Index(['customerid', 'subscription_start_date', 'subscription_type',
'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
'churn flag', 'customer_name', 'country', 'state', 'gender', 'dob',
'interests', 'pincode', 'complaint_date', 'escalations', 'csat_score',
'complaint_count', 'customer_age'],
dtype='str')
```

```
In [112... #avg customer tenure
#count of days user hans used our services :cancelltion date else current date
```

```

today=pd.Timestamp.today()
df['tenure_days']=np.where(
    df['cancellation_date'].notna(),
    (df['cancellation_date']-df['subscription_start_date']).dt.days,
    (today-df['subscription_start_date']).dt.days
)
avg_tenure=df['tenure_days'].mean()
print("Avg Tenure (Days)= ",round(avg_tenure),0)

```

Avg Tenure (Days)= 1541 0

```

In [113... #revenue lost from churn user
revenue_at_risk=df.loc[df['churn_flag']==1,'monthly_charges'].sum()
print('revenue_at_risk(Rs in k)=',revenue_at_risk)

```

revenue_at_risk(Rs in k)= 73.94

```

In [114... #Escalation Rate
escalation_rate=(df['escalations']=='Y').mean()*100
print("escalation_rate=",round(escalation_rate,2),"%")

```

escalation_rate= 19.05 %

```

In [115... #average complain per user
avg_complain=df['complaint_count'].sum()/df['customerid'].nunique()
print("avg_complain per user =",round(avg_complain,2))

```

avg_complain per user = 0.43

```

In [116... df.columns

```

```

Out[116... Index(['customerid', 'subscription_start_date', 'subscription_type',
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
      'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
      'interests', 'pincode', 'complaint_date', 'escalations', 'csat_score',
      'complaint_count', 'customer_age', 'tenure_days'],
      dtype='str')

```

```

In [138... #corelation escalation and churn
#df['escalations']=np.where(df['escalations']=='Y',1,0) # encoding string to int ty
corr_df=df[['escalations','churn_flag']].dropna()

correlation=corr_df['escalations'].corr(df['churn_flag'])
print('corelation escalation and churn is',round(correlation,2))

```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[138], line 5
      1 #corealation escalation and churn
      2 #df['escalations']=np.where(df['escalations']=='Y',1,0) # encoding string to
int type
      3 corr_df=df[['escalations','churn flag']].dropna()
      4
----> 5 correlation=corr_df['escalations'].corr(df['churn flag'])
      6 print('corealation escalation and churn is',round(correlation,2))
      7

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\pandas\core\series.
py:2770, in Series.corr(self, other, method, min_periods)
    2767 if len(this) == 0:
    2768     return np.nan
-> 2770 this_values = this.to_numpy(dtype=float, na_value=np.nan, copy=False)
    2771 other_values = other.to_numpy(dtype=float, na_value=np.nan, copy=False)
    2773 if method in ["pearson", "spearman", "kendall"] or callable(method):

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\pandas\core\base.p
y:664, in IndexOpsMixin.to_numpy(self, dtype, copy, na_value, **kwargs)
    574 """
    575 A NumPy ndarray representing the values in this Series or Index.
    576
    (...) 661         dtype='datetime64[ns]')
    662 """
    663 if isinstance(self.dtype, ExtensionDtype):
--> 664     return self.array.to_numpy(dtype, copy=copy, na_value=na_value, **kwarg
s)
    665 elif kwargs:
    666     bad_keys = next(iter(kwargs.keys()))

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\pandas\core\arrays
\numpy_.py:589, in NumpyExtensionArray.to_numpy(self, dtype, copy, na_value)
    586     result = result.view()
    587     result.flags.writeable = False
--> 589 result = np.asarray(result, dtype=dtype)
    591 if copy and result is self._ndarray:
    592     result = result.copy()

ValueError: could not convert string to float: 'Y'

```

```

In [118... #churn risk-create a column using existing column
conditions=[
    (df['churn_score']<50),
    (df['churn_score']>=50)&(df['churn_score']<70),
    (df['churn_score']>=100)
]
choices=['low','med','high']
df['churn_risk']=np.select(conditions,choices,default='unknown')
df.columns
df[['churn_risk','churn_score']]

```

```
Out[118... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>churn_risk</th> <th>churn_score</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th>
<th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th>
<th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th>
<th>20</th> </tbody> <tbody> </tbody>
```

low	12
-----	----

unknown	91
---------	----

low	34
-----	----

low	8
-----	---

unknown	88
---------	----

low	22
-----	----

unknown	79
---------	----

low	5
-----	---

low	34
-----	----

low	41
-----	----

low	14
-----	----

unknown	83
---------	----

low	19
-----	----

unknown	76
---------	----

low	3
-----	---

med	58
-----	----

med	62
-----	----

low	27
-----	----

unknown	99
---------	----

low	7
-----	---

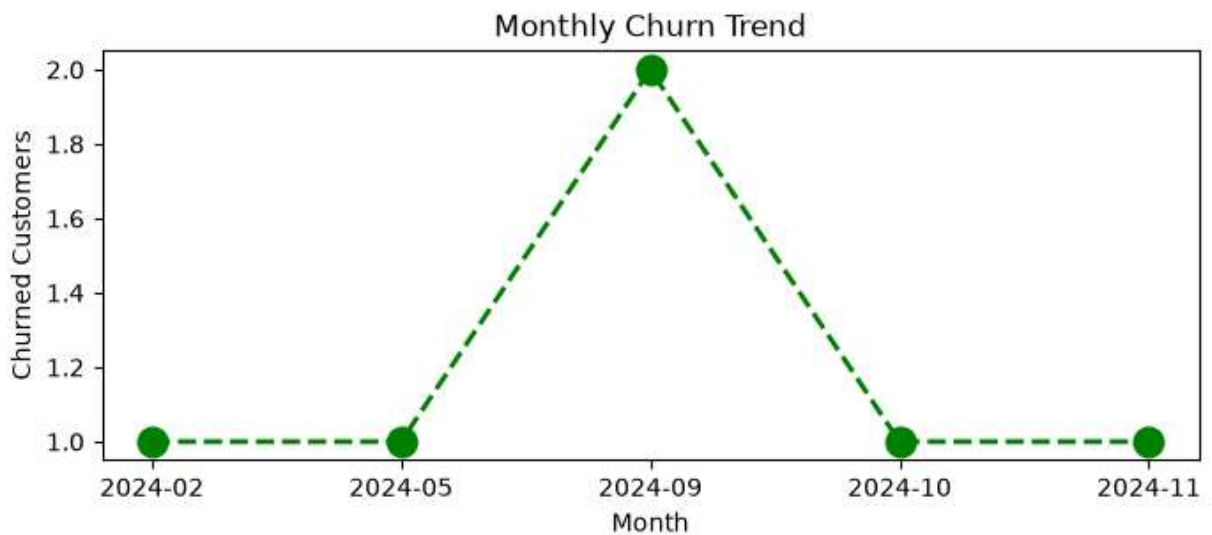
low	47
-----	----

```
In [119... df_visual=df.copy()
```

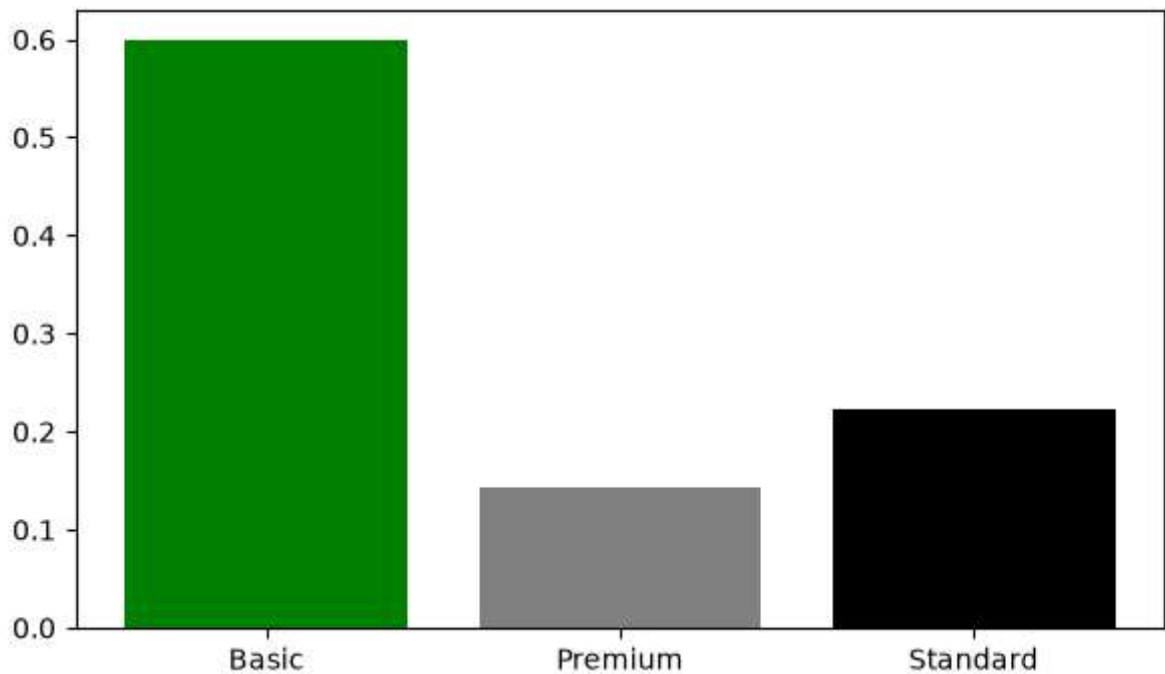
```
In [120... df_visual.columns
```

```
Out[120...] Index(['customerid', 'subscription_start_date', 'subscription_type',
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
      'churn flag', 'customer_name', 'country', 'state', 'gender', 'dob',
      'interests', 'pincode', 'complaint_date', 'escalations', 'csat_score',
      'complaint_count', 'customer_age', 'tenure_days', 'churn_risk'],
      dtype='str')
```

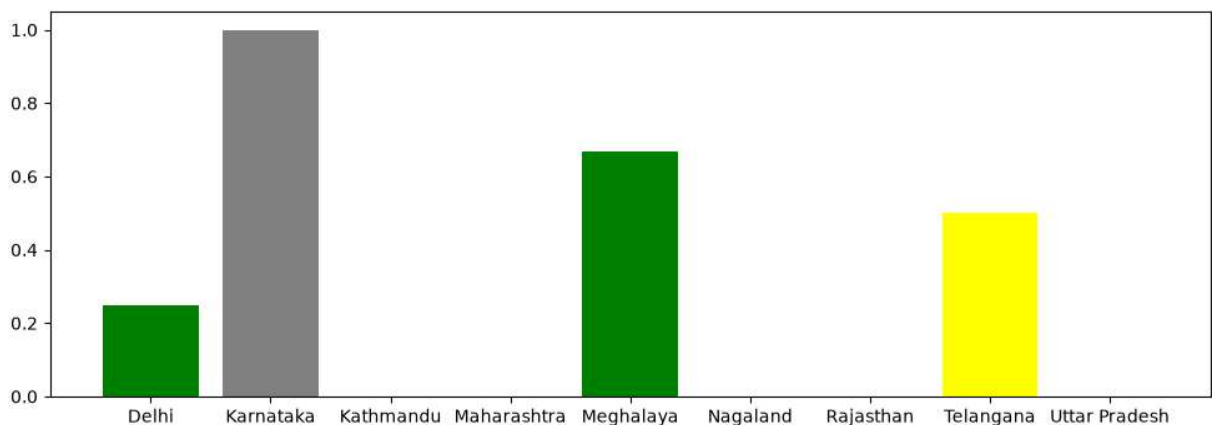
```
In [121...] #monthly churn trend (Time series KPI)
df_visual['cancellation_Month']=df_visual['cancellation_date'].dt.to_period('M')
churn_trend=df_visual[df_visual['churn flag']==1].groupby('cancellation_Month').size
plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str),churn_trend.values,color='green', marker='o')
plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.show()
```



```
In [122...] #churn by plan type
churn_plan=df_visual.groupby('plan_type')['churn flag'].mean()
plt.figure(figsize=(7,4))
colors=['green','gray','black']
plt.bar(churn_plan.index,churn_plan.values,color=colors)
plt.show()
```



```
In [123... #churn bye states
churn_plan=df_visual.groupby('state')['churn flag'].mean()
plt.figure(figsize=(12,4))
colors=['green','gray','red','yellow']
plt.bar(churn_plan.index,churn_plan.values,color=colors)
plt.show()
```



```
In [135...
```

```
In [133... df_visual[['plan_type','contract_type','churn_score','churn flag','churn_risk','esc
```

Out[133... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th> <th>churn
flag</th> <th>churn_risk</th> <th>escalations</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th>
<th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th>
<th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th>
<th>20</th> </tbody> <tbody> </tbody>

Standard	Annual	12	0	low	NaN
Premium	Annual	91	1	unknown	Y
Basic	Monthly	34	0	low	NaN
Premium	Annual	8	0	low	NaN
Standard	Monthly	88	1	unknown	Y
Standard	Annual	22	0	low	N
Basic	Monthly	79	1	unknown	N
Premium	Annual	5	0	low	NaN
Standard	Monthly	34	0	low	NaN
Basic	Annual	41	0	low	NaN
Premium	Annual	14	0	low	NaN
Standard	Monthly	83	1	unknown	Y
Standard	Annual	19	0	low	NaN
Basic	Monthly	76	1	unknown	Y
Premium	Annual	3	0	low	NaN
Standard	Monthly	58	0	med	NaN
Premium	Annual	62	0	med	NaN
Standard	Annual	27	0	low	NaN
Basic	Monthly	99	1	unknown	N
Premium	Annual	7	0	low	NaN
Standard	Monthly	47	0	low	NaN

In [126... df_encoded.head()

```
Out[126... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th> <th>churn
flag</th> <th>churn_risk</th> <th>escalations</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>
```

```
2 0 12 0 0 NaN
```

```
1 0 91 1 2 Y
```

```
0 1 34 0 0 NaN
```

```
1 0 8 0 0 NaN
```

```
2 1 88 1 2 Y
```

```
In [159... df_encoded = df_visual[
    ['plan_type', 'contract_type', 'churn_score',
     'churn flag', 'churn_risk', 'escalations']
].copy()

categorical_cols = [
    'plan_type',
    'contract_type',
    'churn flag',
    'churn_risk',
    'escalations'
]

for col in categorical_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

```
In [160... df_encoded.dtypes
```

```
Out[160... plan_type      int8
contract_type  int8
churn_score    int64
churn flag     int8
churn_risk     int8
escalations    int8
dtype: object
```

```
In [161... df_encoded.head()
```

```
Out[161... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th> <th>churn
flag</th> <th>churn_risk</th> <th>escalations</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>
```

```
2 0 12 0 0 -1
```

```
1 0 91 1 2 1
```

```
0 1 34 0 0 -1
```

```
1 0 8 0 0 -1
```

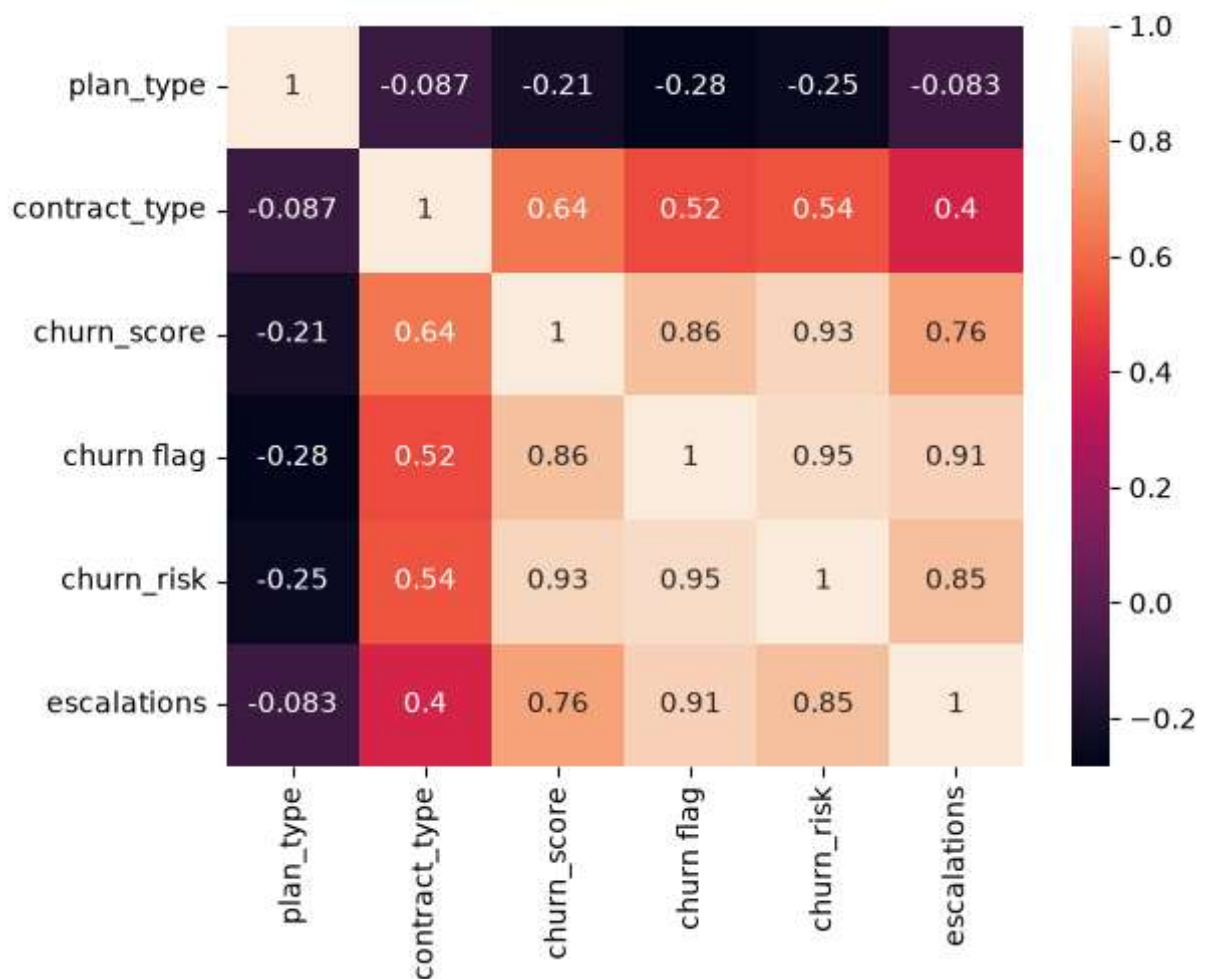
```
2 1 88 1 2 1
```

```
In [162... df_encoded.select_dtypes(include='object').columns
```

```
Out[162... Index([], dtype='str')
```

```
In [163... sns.heatmap(df_encoded.corr(), annot=True)
```

```
Out[163... <Axes: >
```



```
In [173... #correct method of encoding-bases on priority
df_encoded = df_visual[
    ['plan_type', 'contract_type', 'churn_score',
     'churn_flag', 'churn_risk', 'escalations']
].copy()

order_mapping = {
    'plan_type':['Basic', 'Standard', 'Premium'],
    'contract_type':['Monthly', 'Annual'],
    'churn_flag':['0', '1'],
    'churn_risk':['low', 'med'],
    'escalations':['Y', 'N'],

}

for col in order_mapping:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

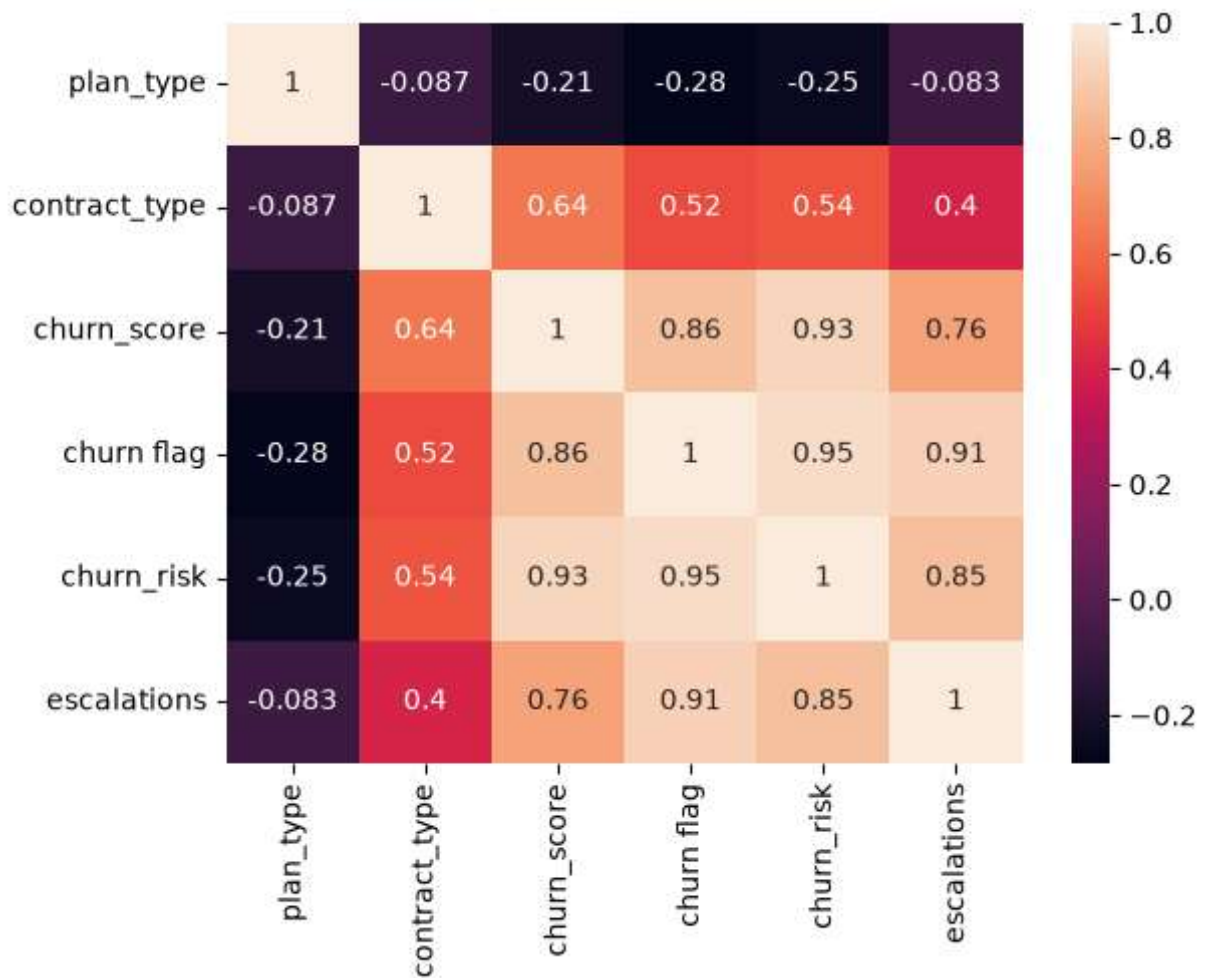
```
In [172... df_visual['churn_risk'].unique()
```

```
Out[172... <StringArray>
['low', 'unknown', 'med']
Length: 3, dtype: str
```

```
In [ ]:
```

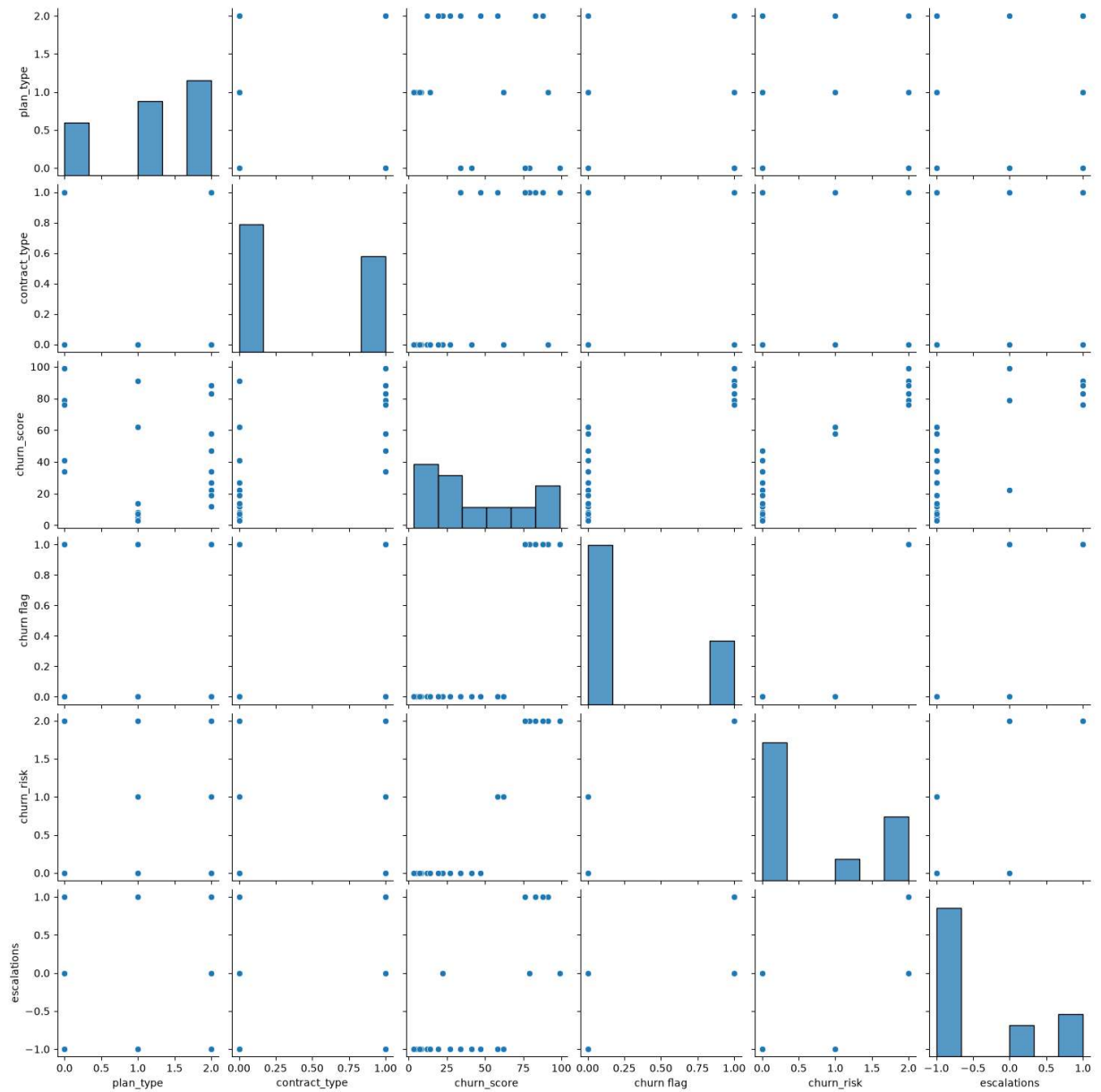
```
In [174... sns.heatmap(df_encoded.corr(), annot=True)
```

```
Out[174... <Axes: >
```



In [176... `sns.pairplot(df_encoded)`

Out[176... `<seaborn.axisgrid.PairGrid at 0x20fd8769e50>`

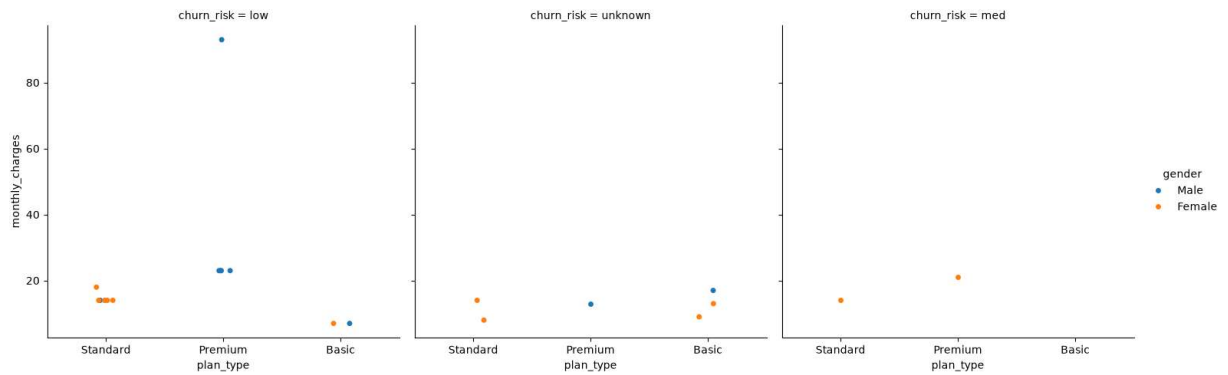


In [180...

```
sns.catplot(
    data=df_visual,
    x='plan_type',
    y='monthly_charges',
    hue='gender',
    col='churn_risk'
)
```

Out[180...

```
<seaborn.axisgrid.FacetGrid at 0x20fd1f73620>
```



In [178... `df_visual.columns`

Out[178... `Index(['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'churn flag', 'customer_name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode', 'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'customer_age', 'tenure_days', 'churn_risk', 'cancellation_Month'], dtype='str')`

In []:

In [183... `pd.pivot_table(df_visual, index='plan_type', values='monthly_charges', aggfunc='mean')`

Out[183... `<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>monthly_charges</th> <th>plan_type</th> <th></th> </thead> <tbody> <th>Basic</th> <th>Premium</th> <th>Standard</th> </tbody> <tbody></tbody>`

10.590000

31.275714

13.767778

In [184... `pd.pivot_table(df_visual, index='plan_type', values=['monthly_charges', 'customerid', 'churn flag'], aggfunc={'monthly_charges': 'sum', 'customerid': 'nunique', 'churn flag': 'mean'})`

```
Out[184... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>churn flag</th> <th>customerid</th> <th>monthly_charges</th>
<th>plan_type</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>Basic</th>
<th>Premium</th> <th>Standard</th> </tbody> <tbody> </tbody>

0.600000  5   52.95
0.142857  7  218.93
0.222222  9  123.91
```

```
In [ ]: #working with sql in python(pandas)
```

```
In [185... #create db in sql
conn=sqlite3.connect('test_database.sqlite')
#table details
conn.execute("CREATE TABLE users(first_name TEXT, country TEXT, budget INTEGER)")
#COMMIT AND SAVE
conn.commit()
```

```
In [206... #insert data
cursor=conn.cursor()
#write sql query to insert records in sql table
cursor.execute(
    """
    INSERT INTO users VALUES
    ('MADHAV','INDIA','5000'),
    ('SAHIL','INDIA','3000'),
    ('PRERNA','INDIA','2000')

    """
)
#commit and save
conn.commit()
print("data print succesfully")
```

data print succesfully

```
In [207... #check inserted data in table
conn=sqlite3.connect('test_database.sqlite')
query="""SELECT * FROM users"""
df_results=pd.read_sql(query,conn)
df_results.head()
```

Out[207... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>first_name</th> <th>country</th> <th>budget</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>

MADHAV	INDIA	5000
--------	-------	------

SAHIL	INDIA	3000
-------	-------	------

PRERNA	INDIA	2000
--------	-------	------

MADHAV	INDIA	5000
--------	-------	------

SAHIL	kNDIA	3000
-------	-------	------

In [209... *#aggragation*
 query="""
 SELECT first_name,sum(budget)as total_budget
 FROM users
 GROUP BY country
 """"
 df_agg=pd.read_sql(query,conn)
 df_agg

Out[209... <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>first_name</th> <th>total_budget</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> </tbody> <tbody> </tbody>

MADHAV	15000
--------	-------

SAHIL	3000
-------	------

PRERNA	2000
--------	------

In [210... *#always close the with db once the table is over*
 conn.close()